MCPForge — Local Notes & Tasks MCP Server

> **Short description:** A real Model Context Protocol server in Python: a local "notes & tasks" capability exposed as 2 Tools, 1 Resource and 1 Prompt over stdio, backed by SQLite, with input validation and a standalone client that discovers and uses everything at runtime. Any MCP client (Claude Desktop, an IDE, your own agent) can connect with zero custom integration.

Tech: Python · official `mcp` SDK (MCPServer) · SQLite · MCP Inspector · **Type:** MCP server + client (no LLM call inside) · **Level:** Intermediate (bonus)

Demo



Overview

MCPForge exposes a small "Local Notes & Tasks" capability — add notes, search them, read them, summarize them — over one standard protocol instead of hardcoded into a single app. Any MCP-compatible client connects, discovers what the server offers at runtime

(`list_tools` / `list_resources` / `list_prompts`), and calls whatever it needs by name, with no knowledge of the server's internals. There is no LLM call anywhere inside the server — this is about the protocol, and about getting the Tool / Resource / Prompt distinction right.

Why

Without a shared protocol, every app that wants your notes/tasks logic needs its own hand-written integration, and separate copies drift apart the moment one changes. MCPForge builds the logic once, behind MCP, so one implementation serves every client. MCP isn't a speed win — a direct function call is faster — it's a *sharing* win: different clients and languages use one server.

Features

- **Two Tools** — `add_note` (write) and `search_notes` (case-insensitive query), backed by a real SQLite database that survives a restart.
- **One Resource** — `notes://all` , a read-only listing, kept separate from Tools because reading it changes nothing.
- **One Prompt** — `summarize_notes` , a reusable parameterized template, not a string hardcoded into one app.
- **Clean input validation** — empty or whitespace-only notes are rejected with a clear, client-readable error instead of being silently stored.
- **A standalone client** — `client.py` connects, discovers tools/resources/prompts, and exercises all three building blocks end to end.
- **Inspector-ready** — testable by hand with the official MCP Inspector (`mcp dev server.py`) before any client code exists.
- **stdout-safe logging** — all logging goes to stderr, because on the stdio transport stdout is the protocol channel.

Demo

```
$ python client.py
Tools: ['add_note', 'search_notes']
Resources: ['notes://all']
Prompts: ['summarize_notes']
add_note: Note added with id 1.
search_notes: Buy milk and eggs
notes://all: [1] Buy milk and eggs (2026-09-11T10:03:12.481022)
summarize_notes: Using the attached notes (see notes://all), summarize the
empty note correctly rejected: Error: note text cannot be empty or whitespace.
```

Architecture

```
client (Claude Desktop / client.py / MCP Inspector)
   │  stdio
   ▼
server.py  (MCPServer "notes-server")
   ├── @mcp.tool()     add_note(text)      ──► write + validate
   ├── @mcp.tool()     search_notes(query) ──► case-insensitive search
   ├── @mcp.resource() notes://all         ──► read-only listing
   └── @mcp.prompt()   summarize_notes()   ──► a filled-in template
                    │
                    ▼
             src/db.py  ──►  notes.db (SQLite)
```

- **Discover, don't hardcode** — the client calls `list_tools/resources/prompts` ; it never imports from `server.py` .
- **Tool vs. Resource vs. Prompt** — a write/query is a Tool; a read-only listing is a Resource; a reusable template is a Prompt.
- **No LLM inside** — the Prompt returns *text*; whichever client holds it sends it to a model.

Project Structure

```
<repo folder>/   (MCPForge-Local-Notes-And-Tasks-Server/)
├── src/
│   ├── utils/
│   │   ├── __init__.py
│   │   └── logger.py          get_logger() -> stderr
│   ├── __init__.py
```

```
│   ├── config.py             Config, load_config()
│   ├── db.py                 init_db/insert/find/list_all
│   └── exceptions.py         MissingConfigError, EmptyNoteError
├── tests/
│   ├── integration/
│   │   └── test_server.py    drives the server via a client
│   └── unit/
│       ├── test_config.py    no server needed
│       └── test_db.py        no server needed
├── .env                      LOG_LEVEL, NOTES_DB_PATH — never committed
├── .env.example              the setting names, no secrets
├── .gitignore                keeps .env, .venv/, notes.db out of git
├── client.py                 the standalone MCP client
├── LICENSE                   lets others reuse your code (MIT)
├── README.md                 the repo's front page
├── requirements.txt          the libraries, for pip install -r
└── server.py                 the MCP server: 2 Tools, 1 Resource, 1 Prompt
```

Getting Started

Requirements: Python 3.10+. No API key — this server calls no model.

```
git clone <this-repo-url>
cd MCPForge-Local-Notes-And-Tasks-Server
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env
```

`pip install -r requirements.txt` brings the `mcp[cli]` SDK and the MCP Inspector.

Configuration

Variable	Required	Default	Meaning
`LOG_LEVEL`	no	`INFO`	`DEBUG`, `INFO`, `WARNING` or `ERROR`
`NOTES_DB_PATH`	no	`notes.db`	Where the SQLite notes file lives

Setting	Where	Default
Server name	`server.py` → `MCPServer("notes-server")`	`notes-server`
Database file	`.env` → `NOTES_DB_PATH`	`notes.db`

Usage

Run from the repo root:

```
mcp dev server.py       # the MCP Inspector, to call tools by hand
python client.py        # the standalone client: discover + use everything
```

To connect from Claude Desktop or another MCP client, point it at `python server.py` from this folder.

What the server exposes	Kind	What it does
`add_note(text)`	Tool	Stores a note; rejects empty text with a clear error
`search_notes(query)`	Tool	Case-insensitive substring search over stored notes
`notes://all`	Resource	A read-only listing of every note, oldest first
`summarize_notes(start_date, end_date)`	Prompt	A filled-in template for a model to summarize notes in a date range

Running the Tests

Run from the repo root:

```
# unit tests — no server, no network
python -m tests.unit.test_config
python -m tests.unit.test_db

# integration test — launches server.py via a real MCP client
python -m tests.integration.test_server
```

The unit tests check the settings and the SQLite helpers. The integration test launches the server and, through a real client, confirms the tools are listed, both tools work, an empty note is rejected, the Resource lists notes, and the Prompt is a filled-in template.

Error Handling

Situation	What happens
Empty or whitespace-only note text	`add_note` returns `"Error: note text cannot be empty..."` — never stored
Unknown search query	`search_notes` returns an empty list, not an error
Reading `notes://all` on a fresh database	Returns `"No notes yet."`, not a silent empty string
A stray `print()` in server code	Would corrupt the stdio protocol — all logging goes to stderr instead
The client can't launch the server	`client.py` prints one clear line, not a raw async traceback

Design Decisions

- **Tool vs. Resource vs. Prompt by purpose** — a write/query is a Tool; a read-only listing is a Resource; a reusable template is a Prompt. Cramming everything into Tools is the real mistake this project avoids.
- **Catch-and-return on bad input** — a raised exception reaches the client only as a generic message, so `add_note` catches `EmptyNoteError` and returns a clear string.
- **Logging to stderr** — on stdio, stdout is the protocol channel; a log line there breaks the connection.
- **SQLite over an in-memory list** — notes must survive a restart for a real client to rely on them.
- **A real client, not just the Inspector** — `client.py` proves the exact code path a future agent uses, discovery and all.

Limitations and Roadmap

- One Resource and one Prompt — enough to learn the distinction; a real server grows more.
- stdio transport only — fine for a local server and for Claude Desktop; a hosted server would add an HTTP transport.
- No auth — a local, personal server.
- Possible next steps: a tasks table alongside notes, a per-note Resource (`notes://{id}`), an HTTP transport, and using this server from a multi-agent client (Project 11).

License

MIT — see `LICENSE` .